

ROS 2 CATAMARAN CONTROL SYSTEM

1. SYSTEM OVERVIEW

This ROS 2-based system enables remote, real-time control of a catamaran equipped with T100 thrusters and a Raspberry Pi. The control logic is based on differential thrust, and the system is designed for modularity, safety, and extensibility.

2. HARDWARE ARCHITECTURE

- **Raspberry Pi:** Runs ROS 2, controls ESCs via GPIO (pins 17 and 18).
- **ESCs:** Receive PWM signals from Pi, drive the T100 thrusters.
- **T100 Thrusters:** Provide propulsion and steering.
- **Power:** Pi is powered by a power bank; ESCs/thrusters by a main supply.
- **WiFi:** Connects Pi and remote computer for ROS 2 communication.

3. SOFTWARE ARCHITECTURE

ROS 2 Nodes

Node Name	Location	Function
thruster_controller	Raspberry Pi	Subscribes to /thruster_cmd, outputs PWM
teleop_twist_keyboard	Remote Computer	Publishes velocity commands to /thruster_cmd

Topics

- */thruster_cmd (geometry_msgs/msg/Twist):* Command topic for velocity and turning.

4. CONTROL LOGIC

Keyboard to Motion Mapping

Keyboard Input	linear.x	angular.z	Left Thruster Command	Right Thruster Command	Catamaran Motion
W	1.0	0.0	1.0	1.0	Full forward
S	-1.0	0.0	-1.0	-1.0	Full reverse
A	0.0	1.0	-1.0	1.0	Spin right (CW)
D	0.0	-1.0	1.0	-1.0	Spin left (CCW)
W+A	1.0	1.0	0.0	2.0	Forward + right arc
W+D	1.0	-1.0	2.0	0.0	Forward + left arc
S+A	-1.0	1.0	-2.0	0.0	Reverse + right arc
S+D	-1.0	-1.0	0.0	-2.0	Reverse + left arc
(no key)	0.0	0.0	0.0	0.0	Stop / neutral

- $left_cmd = linear.x - angular.z$
- $right_cmd = linear.x + angular.z$
- PWM output is clamped between PWM_MIN and PWM_MAX .

5. CODE EXPLANATION

A. Thruster Controller Node

Purpose:

Listens to `/thruster_cmd` and converts velocity commands into PWM signals for the left and right ESCs.

Key Sections:

- **Initialization:**
 - Connects to pigpio daemon.
 - Sets ESCs to neutral PWM (safe state).
- **Callback:**
 - For each incoming Twist message, computes left and right thrust using the control logic above.
 - Converts these commands to PWM values and sends to ESCs.
- **Shutdown:**
 - On node destruction, sets both ESCs to neutral for safety.

CODE:

Python

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import pigpio

ESC_LEFT_PIN = 17
ESC_RIGHT_PIN = 18
PWM_NEUTRAL = 1500
PWM_MIN = 1100
PWM_MAX = 1900

class ThrusterController(Node):
    def __init__(self):
        super().__init__('thruster_controller')
        self.pi = pigpio.pi()
        if not self.pi.connected:
            self.get_logger().error("Could not connect to pigpio daemon.")
            raise RuntimeError("pigpio not running")
        self.pi.set_servo_pulsewidth(ESC_LEFT_PIN, PWM_NEUTRAL)
        self.pi.set_servo_pulsewidth(ESC_RIGHT_PIN, PWM_NEUTRAL)
        self.subscription = self.create_subscription(
            Twist,
            '/thruster_cmd',
            self.listener_callback,
            10)
        self.get_logger().info('ThrusterController node started.')
```

```

def listener_callback(self, msg):
    forward = msg.linear.x
    turn = msg.angular.z
    left_cmd = forward - turn
    right_cmd = forward + turn
    left_pwm = int(PWM_NEUTRAL + left_cmd * (PWM_MAX - PWM_NEUTRAL))
    right_pwm = int(PWM_NEUTRAL + right_cmd * (PWM_MAX - PWM_NEUTRAL))
    left_pwm = max(PWM_MIN, min(PWM_MAX, left_pwm))
    right_pwm = max(PWM_MIN, min(PWM_MAX, right_pwm))
    self.pi.set_servo_pulsewidth(ESC_LEFT_PIN, left_pwm)
    self.pi.set_servo_pulsewidth(ESC_RIGHT_PIN, right_pwm)
    self.get_logger().info(f"Set left: {left_pwm}, right: {right_pwm}")

def destroy_node(self):
    self.pi.set_servo_pulsewidth(ESC_LEFT_PIN, PWM_NEUTRAL)
    self.pi.set_servo_pulsewidth(ESC_RIGHT_PIN, PWM_NEUTRAL)
    self.pi.stop()
    super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = ThrusterController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

B. TELEOPERATION NODE

Purpose:

Reads keyboard input and publishes corresponding Twist messages to /thruster_cmd.

How to Use:

- Install:

```
sudo apt install ros-<ros2-distro>-teleop-twist-keyboard
```

- Run:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -
r/cmd_vel:=/thruster_cmd
```

- Use W, A, S, D keys for control as per the logic table above.

C. ROS 2 PACKAGE FILES

package.xml

```
<package format="3">
  <name>catamaran_thruster_control</name>
  <version>0.1.0</version>
  <description>ROS 2 package for catamaran thruster control</description>
  <maintainer email="your@email.com">Your Name</maintainer>
  <license>MIT</license>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
</package>
```

SETUP.PY

python

```
from setuptools import setup

package_name = 'catamaran_thruster_control'

setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Your Name',
    maintainer_email='your@email.com',
    description='ROS 2 package for catamaran thruster control',
    license='MIT',
    entry_points={
        'console_scripts': [
            'thruster_controller = thruster_controller:main',
        ],
    },
)
```

6. WHAT TO RUN WHERE

On the Raspberry Pi (Boat Side):

1. Start pigpio daemon:

```
sudo pigpiod
```

2. Build and source your ROS 2 workspace:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

3. Run the thruster controller node:

```
ros2 run catamaran_thruster_control thruster_controller
```

On the Remote Computer (Operator Side):

1. Source your ROS 2 workspace:

```
cd ~/ros2_ws  
source install/setup.bash
```

2. Run the teleop node:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r  
/cmd_vel:=/thruster_cmd
```

3. Control the catamaran using your keyboard.

7. SYSTEM FLOW

1. **Keyboard input** → *teleop_twist_keyboard* → */thruster_cmd* (Twist message)
2. **Raspberry Pi** → *thruster_controller node* → PWM signals to ESCs
3. **ESCs** → *T100 thrusters* → Catamaran movement

8. SAFETY AND EXTENSIBILITY

- **Safety:** Thrusters are set to neutral on startup and shutdown.
- **Extensibility:** Add autonomy, sensor, or navigation nodes by publishing to */thruster_cmd* or subscribing to other topics.

9. REFERENCES AND BEST PRACTICES

- This approach follows the modular, topic-based design recommended in ROS 2 documentation and by leading aquatic robotics teams.
- Real-time, robust, and extensible for research and advanced applications